

Oculus for Omarchy — Design Ideas

Ideas for a more useful Omarchy shell plugin built on oculus.nvim · 15 September 2026 · Draft for discussion

1. Summary

The current Omarchy plugin uses only one part of oculus.nvim: the tracked-project list. It counts new activity per project and opens a project's feed in Neovim. oculus.nvim's strongest feature is reviewing pull requests, issues and commits locally, in real, editable buffers, and the plugin doesn't use it at all yet.

The ideas below bring that review workflow to the desktop and make the bar's signals worth acting on. They also fit the plugin into how Omarchy already works: workspaces, menus, notifications and keybindings. Suggested first steps: **focus-instead-of-duplicate** (#8), an **inspect prompt** (#1), and **context-aware browser actions** (#2).

2. Where things stand

Part	What it does today
Bar widget <code>andrewgilley.oculus</code>	Eye icon with the total count of new items; a popout lists the top 5 projects by new pushes and merged PRs since you last opened them.
Fetcher <code>oculus-activity</code>	Headless Neovim using oculus's own GitHub/Codeberg clients and your <code>tracking.json</code> . Runs every 15 min, or when the popout opens if the data is more than 5 min old. About 42 requests per run for 21 projects; authenticates with <code>gh</code> .
Bridge <code>oculus_omarchy</code>	Runs inside Neovim. Marks a project seen when you open its feed, and publishes the Neovim socket.
Launcher <code>oculus-open-project</code>	Clicking a project opens Ghostty on an empty workspace, with Neovim showing that project's feed through <code>require("oculus").open_project()</code> .

3. Ideas

A. Bring oculus's review tools to the desktop

1. Inspect from anywhere

A keybind (for example `SUPER+0`) opens a small prompt, like Omarchy's emoji and image pickers. Paste or type `neovim/neovim#123`, a PR/issue URL or a commit SHA, and it opens Ghostty + Neovim on `:OculusInspect` for that target.

Needs: an overlay plugin and a launcher like `oculus-open-project`. **oculus changes:** none; `oculus.inspect()` is already public. **API cost:** none beyond the inspection itself.

2. Context-aware browser actions

The widget notices when the focused window is a GitHub or Codeberg page and offers the action that fits, as a small button next to the eye icon that only appears on those pages. This replaces the earlier "add from clipboard" idea and most of the "copy a URL → Inspect" idea.

Page	Typical GitHub title (verify during implementation)	Action
Repository	GitHub - owner/repo: description, Issues · owner/repo	Track in Oculus, or "✓ tracked"
User / org	username (Full Name) · GitHub	Track user (an API check tells a user from an org)
Pull request	Title by author · Pull Request #123 · owner/repo	Inspect in Oculus
Issue	Title · Issue #123 · owner/repo	Inspect in Oculus
Commit	message · owner/repo@abc1234	Inspect in Oculus

How it works: Omarchy's own active-window widget already reads the focused window's title and app id from `ToplevelManager.activeToplevel`, and the title updates as you change windows or tabs, so no polling is needed. Codeberg's title formats still need to be collected.

Safeguards: only show the button when the title clearly matches. Confirm the repo or user exists with one API call before adding it, and write it through oculus's own tracking-file validation. Titles don't contain the URL; if matching proves unreliable, a small Chromium extension could send the real URL, but that is more to build and maintain.

Privacy: the widget reads every focused window's title. Nothing leaves the machine, and it only acts on GitHub/Codeberg matches, but the plugin should say so. **oculus changes:** a public "add to tracking" function; `tracking.lua` already has validation and a `mutate` helper to build it on.

3. Item-level rows

Expand a project in the panel to see its recent PRs and commits, and click one to inspect it directly instead of only opening the feed.

Needs: the fetcher to keep the top few items per project; it already downloads them. **oculus changes:** none.

B. Make the counts actionable

4. Review queue

A section, or the bar count itself, for PRs waiting on your review and issues assigned to you across tracked projects. "2 PRs need your review" is something to act on; "12 new commits in linux" is only interesting.

Needs: `gh` authentication (already present). **API cost:** roughly 1–2 search requests per refresh; GitHub's search API has its own, lower rate limit.

5. Notification digests

One notification per refresh through Omarchy's notification service, for example "neovim: 3 PRs merged, 1 release", instead of one popup per event. Clicking opens that project's feed. Each project can be set to notify or to count only.

API cost: none; it reuses the existing fetch. **Care:** must not notify about the same event twice (use `seen.json`).

6. Releases

Watch tags and releases for tracked projects (Hyprland, Neovim nightly) and show them in the style of Omarchy's update widget.

API cost: 1 request per project, and it can run less often than the activity check.

7. People

oculus already tracks users. A "People" tab would show what they've been doing, such as dhh's recent Omarchy commits.

API cost: 1 request per tracked user (4 today).

C. Fit it into Omarchy's workflow

8. Focus instead of duplicate

Give each project its own app id and launch through `omarchy-launch-or-focus`. Clicking a project a second time jumps to its existing workspace instead of opening another window.

Size: small; a change to `oculus-open-project` only.

9. Workspaces for fixes

oculus can already start a `git worktree` from an issue. Pair it with its own Hyprland workspace, and list active inspections and worktrees in the bar so you can jump between them.

oculus changes: a public entry point for starting a worktree, and a way to list open inspections.

10. Omarchy menu entries

An Oculus submenu: open a project, inspect, refresh, mark all seen. Added through `~/.config/omarchy/extensions/omarchy-menu.jsonc`, so it matches how Omarchy users already find things.

Size: small; no QML needed.

11. AI weekly digest

Once a day, use oculus's existing Codex or Antigravity integration to write a short "what changed in your projects this week" summary for the panel.

Care: slower and costs model usage, so it should be cached and optional. **oculus changes:** a headless entry point for the summary.

4. Suggested order

Phase	Ideas	Why
1	#8 Focus instead of duplicate	Small, and fixes the current duplicate-window behaviour right away.
2	#1 Inspect from anywhere, #2 Context-aware browser actions	Use oculus's strongest feature; #1 needs no oculus changes.
3	#4 Review queue, #5 Notification digests	Turn a passive count into things that need you.
Later	#3, #6, #7, #10, then #9 and #11	Useful additions; #9 and #11 need new oculus APIs.

5. Costs and risks

- **API budget.** Each new data source adds requests on every refresh. Before adding many, use conditional requests (ETags), so quiet repos don't count against the rate limit, and let slower sources such as releases refresh less often.
- **oculus.nvim API surface.** Several ideas need new public functions, as `open_project` did (commit `1d4a0a6`). Adding them in oculus keeps the widget from depending on internal state.
- **Replacing the bridge's polling.** `User` autocmds from oculus (for example when a feed loads or an inspection opens) would replace the bridge's 5-second polling and make the widget respond immediately.
- **Multiple Neovim instances.** The snapshot is last-writer-wins today. Keying it by Neovim pid becomes important once the widget focuses or controls a specific instance.
- **Title matching is a best guess (#2).** Keep it conservative, and keep the browser extension as a fallback rather than the first step.

6. Open questions

- Should the bar count show new activity, the review queue, or both, and which one wins when space is tight?
- Which projects deserve notifications, and should that be set per group in `tracking.json`?
- Inspect in a new Ghostty window, or reuse the Neovim you already have open when there is one?
- Is a browser extension acceptable later if title matching isn't reliable enough?